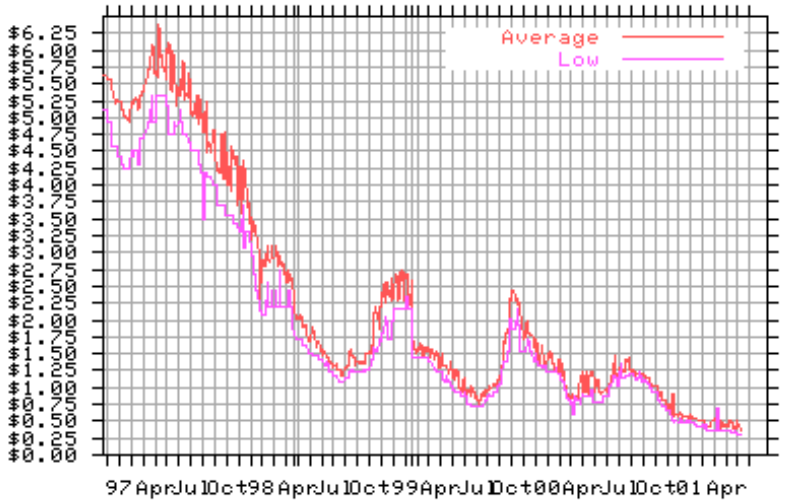
Graphs

Slides courtesy: Douglas W. Harder (through Mausam),
Ragesh Jaiswal

What are graphs?

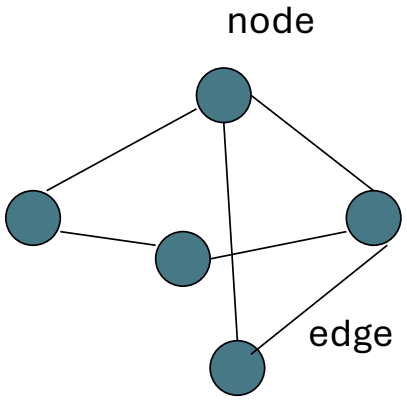
- Yes, this is a graph….



- But we are interested in a different kind of “graph”

Graphs

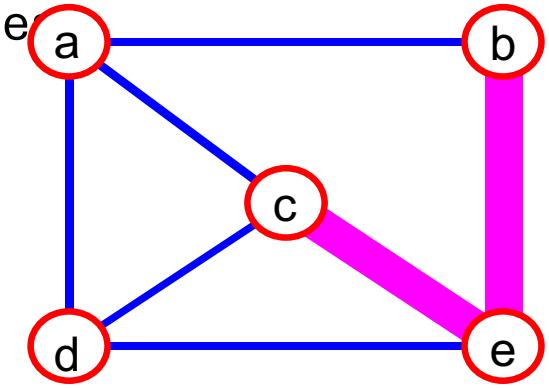
- Graphs are composed of
  - Nodes (vertices)
  - Edges (arcs)



- Nodes
  - Labeled or unlabeled
- Edges
  - Directed or undirected
  - Labeled or unlabeled

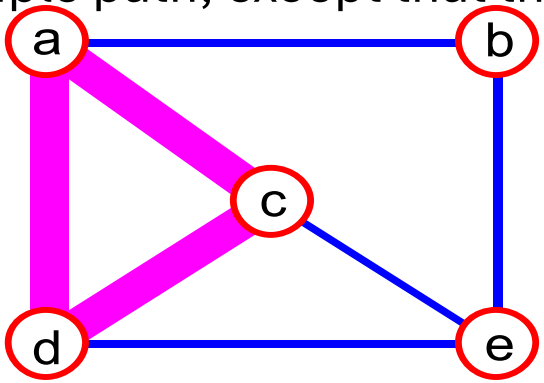
More Graph Terminology

- simple path: no repeated vertices

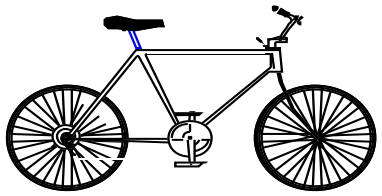


b e c

- cycle: simple path, except that the last vertex is the same as the first vertex

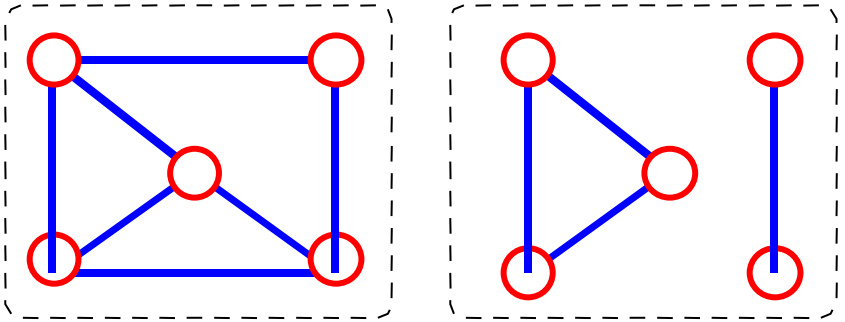


a c d a

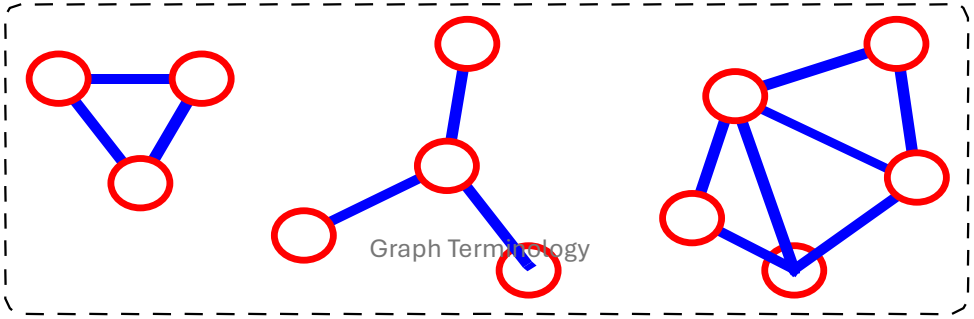


Even More Terminology

- connected graph: any two vertices are connected by some path

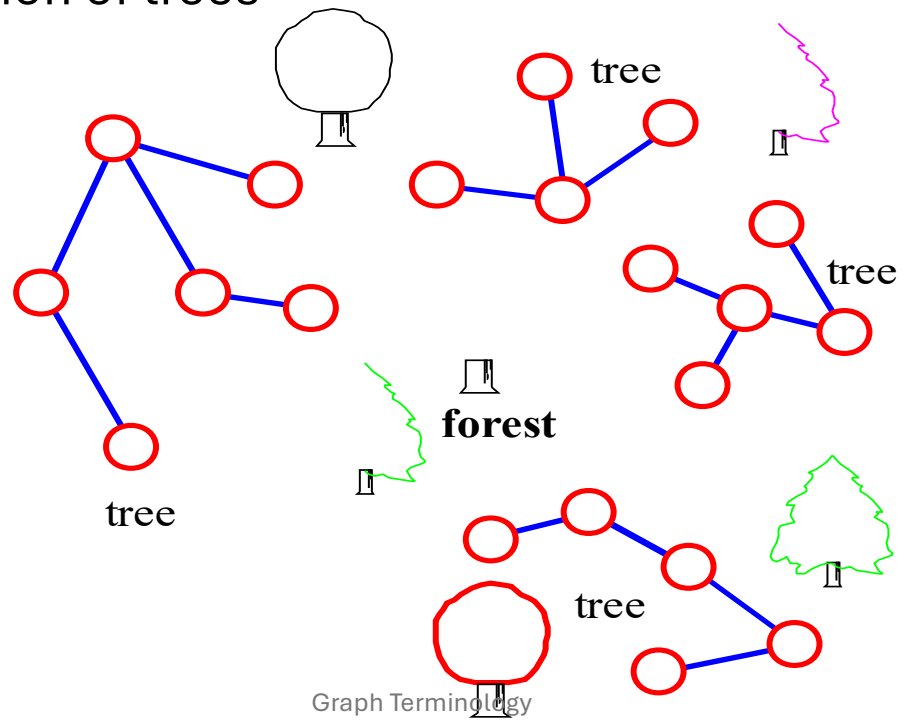


- subgraph: subset of vertices and edges forming a graph
- connected component: maximal connected subgraph. E.g., the graph below has 3 connected components.



Trees from Perspective of Graphs

- (free) tree - connected graph without cycles
- forest - collection of trees



Handshaking Theorem

- Let G=(V,E) be an undirected graph with |E|=e edges. Then

$$2e = \sum_{v \in V} \deg(v)$$

Add up the degrees of all vertices.

- Every edge contributes +1 to the degree of each of the two vertices it is incident with
  - number of edges is exactly half the sum of deg(v)
  - the sum of the deg(v) values must be even

Searching in graphs

- Find Properties of Graphs
  - Spanning trees
  - Connected components
  - Bipartite structure
  - Biconnected components
- Applications
  - Finding the web graph – used by Google and others
  - Garbage collection – used in Java run time system

Graph Searching Methodology Breadth-First Search (BFS)

- Breadth-First Search (BFS)
  - Use a queue to explore neighbors of source vertex, then neighbors of neighbors etc.
  - All nodes at a given distance (in number of edges) are explored before we go further

Breadth-First Search

- A Breadth-First Search (BFS) traverses a connected component of a graph, and in doing so defines a spanning tree with several useful properties.
- The starting vertex *s* has level 0, and defines that point as an "anchor."
- In the first round, all of the nodes that are only one edge away from the anchor are visited.
- These nodes are placed into level 1
- In the second round, all the new nodes that can be reached by one edge from level 1 nodes are visited and placed in level 2.
- This continues until every vertex has been assigned a level.
- The label of any vertex ***v*** corresponds to the length of the shortest path from ***s*** to ***v***.

Properties of BFS

- Proposition: Let G be an undirected graph on which a a BFS traversal starting at vertex ***s*** has been performed. Then
  - The traversal visits all vertices in the connected component of ***s*** .
  - The discovery-edges form a spanning tree T, which we call the BFS tree, of the connected component of ***s***
  - For each vertex v at level i, the path of the BFS tree T between s and v has i edges, and any other path of G between ***s*** and v has at least i edges.
  - If (u, v) is an edge that is not in the BFS tree, then the level numbers of u and v differ by at most one.
- Proposition: Let G be a graph with n vertices and m edges. A BFS traversal of G takes time O(n + m). Also, there exist O(n + m) time algorithms based on BFS for the following problems:
  - Testing whether G is connected.
  - Computing a spanning tree of G
  - Computing the connected components of G
  - Computing, for every vertex v of G, the minimum number of edges of any path between ***s*** and v.

BFS Properties

- **Proposition**: Let G be an undirected graph on which a BFS traversal starting at a vertex *s* has been preformed. Then:
  1. The traversal visits all vertices in the connected component of *s*
  2. The discovery edges form a spanning tree of the connected component of *s*

- **Justification of 1**:
  - Let's use a contradiction argument: suppose there is at least on vertex v not visited and let w be the first unvisited vertex on some path from *s* to v.
  - Because w was the first unvisited vertex on the path, there is a neighbor u that has been visited.
  - But when we visited u we must have looked at edge(u, w). Therefore w must have been visited.

- **Justification of 2**:
  - We only mark edges from when we go to unvisited vertices. So we never form a cycle of discovery edges, i.e. discovery edges form a tree.
  - This is a spanning tree because BFS visits each vertex in the connected component of *s*

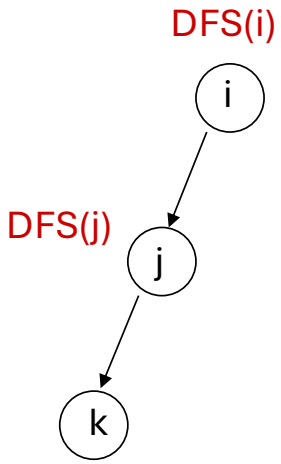
Graph Searching Methodology Depth-First Search (DFS)

- Depth-First Search (DFS)
  - Searches down one path as deep as possible
  - When no nodes available, it backtracks
  - When backtracking, it explores side-paths that were not taken
  - Uses a stack (instead of a queue in BFS)
  - Allows an easy recursive implementation

Depth First Search Algorithm

- Recursive marking algorithm
- Initially every vertex is unmarked

```
DFS(i: vertex)
  mark i;
  for each j adjacent to i do
    if j is unmarked then DFS(j)
end{DFS}
```



Marks all vertices reachable from i

Depth-First Search

Algorithm DFS(v); **Input**: A vertex v in a graph

Output: A labeling of the edges as "discovery" edges and "backedges"

for each edge *e* incident on *v* **do**

if edge *e* is unexplored **then** let *w* be the other endpoint of *e*

if vertex w is unexplored **then** label *e* as a discovery edge

recursively call **DFS(*w*)**

else label *e* as a backedge